

Radix-4 Booth-Wallace Multiplier for a bfloat16 FPU

Custom SystemVerilog significand multiplier integrated into a supplied FP16ALT floating-point unit, verified with directed Questa simulations and evaluated through a six-point Synopsys Design Compiler study.

11 / 11

CUSTOM-FPU DIRECTED RESULTS

1.87 ns

CUSTOM CLOCK CONSTRAINT

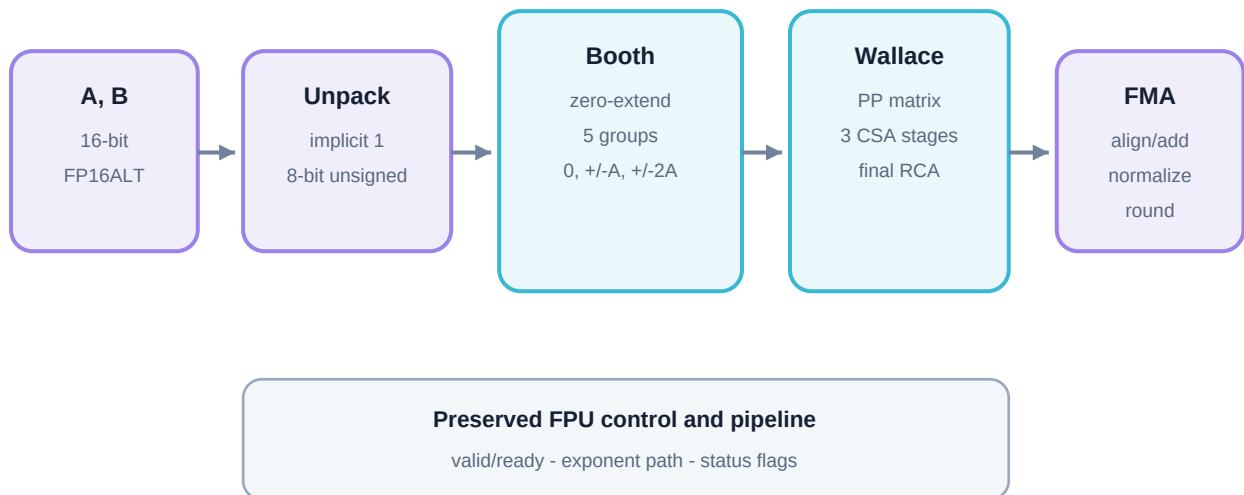
4439.27

CELL AREA (μm^2)

Evidence set: SystemVerilog RTL and testbenches, Questa transcript extracts, Design Compiler timing/area/resource reports, and a reconciled machine-readable results table.

1. Design objective and integration

The starting point is a supplied CVFPU-lite configuration supporting the 16-bit FP16ALT format. The work replaces the behavioral significand multiplication inside `fnew_fma` with a structural unsigned radix-4 Modified Booth multiplier while preserving the FPU interface, exponent path, alignment, normalization, rounding, and valid/ready pipeline.

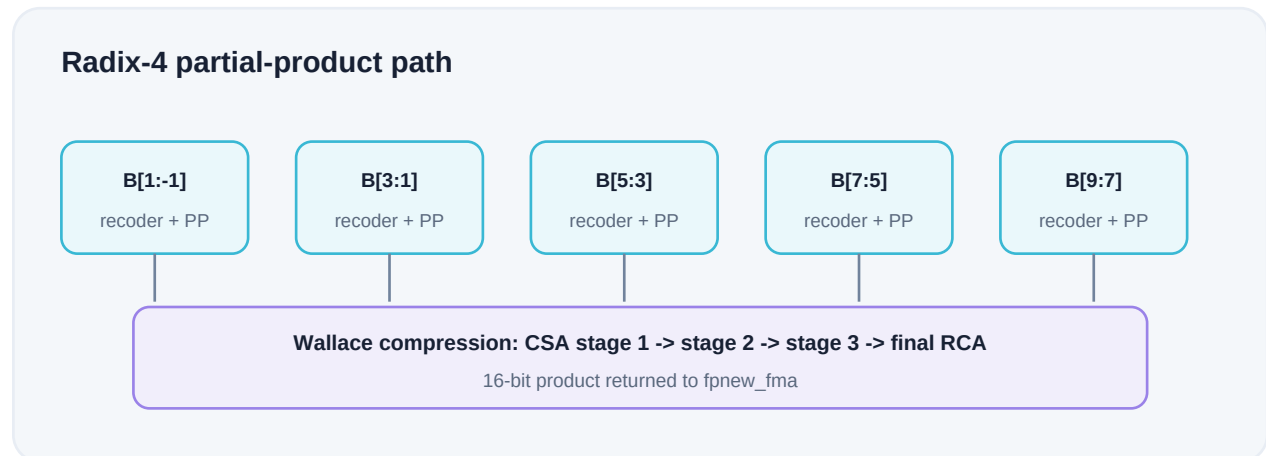


Arithmetic boundary

FP16ALT stores one sign bit, eight exponent bits, and seven fraction bits. For normalized operands, the FPU restores the implicit leading one, creating two unsigned 8-bit significands. The custom block zero-extends each to nine bits for recoding and returns the low-level 16-bit significand product. Operand sign and exponent arithmetic are handled elsewhere in the FPU.

The custom multiplier is an internal FMA datapath replacement, not a complete floating-point multiplier by itself. This distinction keeps sign, exponent, exceptional-value, normalization, and rounding responsibilities clear.

2. Modified Booth and Wallace datapath



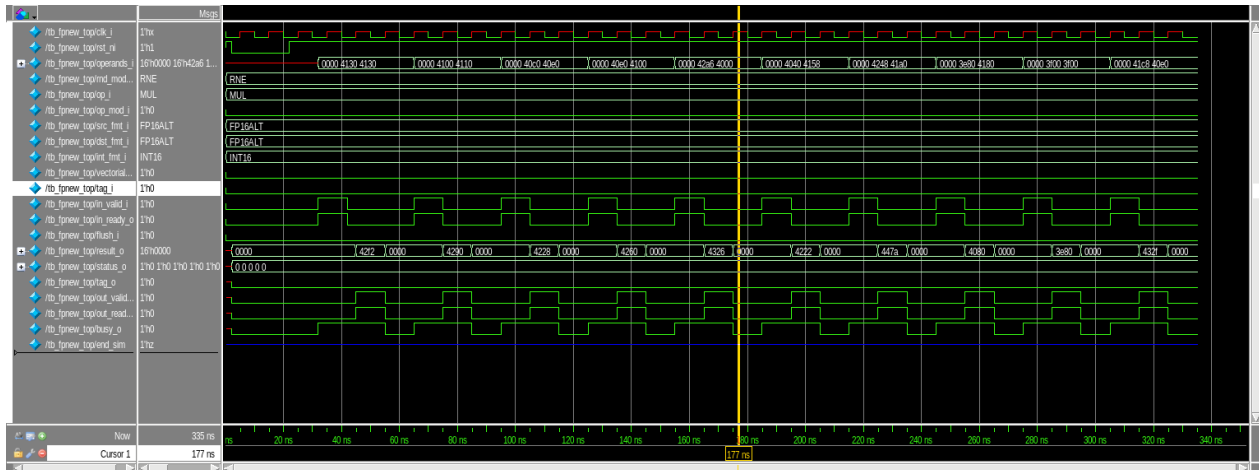
Stage	Implementation
Booth grouping	Five overlapping 3-bit slices, including the implicit low zero and top extension.
Recoding	Each slice selects 0, +A, -A, +2A, or -2A.
Partial products	Generated, extended, and shifted into an 18-bit matrix.
Compression	Three explicit half-adder/full-adder stages reduce the five matrix rows.
Final addition	A ripple-carry adder combines the last two rows; retained low bits form the 16-bit product.

Implementation-specific synthesis result

Negative partial products are expressed with unary negation in multiplier_reduced.sv. The custom resource report identifies five inferred 11-bit DW01_sub resources below the five partial-product generators. The saved implementation is therefore a Booth/Wallace structure, but its partial-product generation is not subtracter-free.

3. Directed functional verification

The SystemVerilog FPU testbench drives one multiplication at a time through the valid/ready interface and logs each bfloat16 result. Reference encodings were generated using the supplied FlexFloat-based utility. The final saved baseline transcript contains 10 matching products; the custom-FPU transcript contains the same 10 plus a negative-result case.



Baseline FPU simulation capture

Configuration	Products	Matching logs	Simulation stop
Supplied FPU multiplier	10	10	1265 ns
Custom Booth-Wallace in FPU	11	11	1365 ns

The saved Questa transcript results were reconciled against the FlexFloat reference encodings.

A	B	Result	A	B	Result
40e0	41c8	432f	4000	42a6	4326
3f00	3f00	3e80	4100	40e0	4260
4180	3e80	4080	40e0	40c0	4228
41a0	4248	447a	4110	4100	4290
4158	4040	4222	4130	4130	42f2
4130	c130	c2f2			

5. Synthesis comparison

All runs use the Nangate Open Cell Library typical view, 0.07 ns clock uncertainty, 0.5 ns maximum I/O delay, and a BUF_X4-equivalent output load. Clock rate is calculated from the capture-clock edge recorded by report_timing. Critical data arrival is shown separately.

Implementation / strategy	Clock (ns)	MHz	Arrival (ns)	Area (um^2)	Slack
Supplied - compile	2.50	400.00	1.93	4431.29	0.00
Supplied - retiming	1.99	502.51	1.89	3941.06	0.00
Supplied - compile_ultra	2.18	458.72	2.08	3548.17	0.00
Supplied - forced CSA	1.95	512.82	1.84	4131.25	0.00
Supplied - forced PPARCH	1.85	540.54	1.74	4212.64	0.00
Custom - Booth-Wallace	1.87	534.76	1.76	4439.27	0.00

Comparison	Clock-rate change	Area change
Custom vs forced PPARCH	-1.1%	+5.4%
Custom vs compile_ultra	+16.6%	+25.1%

Maximum clock rate is derived from each run's configured clock constraint; critical data arrival is reported separately as a path metric.

6. Evidence and tool flow

Evidence

Artifact	Purpose
results/verified_results.csv	Reconciled clock, frequency, path arrival, slack, and cell area.
results/simulation/	Final successful Questa transcript results.
cvfpu_lite/syn/reports/	Five supplied-multiplier timing/area runs and CSA/PPARCH resource reports.
cvfpu_lite/ex3/syn/reports/	Custom multiplier initial and constrained timing/area/resource reports.
cvfpu_lite/ex3/src/	Final multiplier RTL and modified FMA integration.

Tool flow

Reference values were produced by a supplied FlexFloat-based C++ utility. Questa/ModelSim 2020.4 was used for RTL and saved gate-level simulations. Synopsys Design Compiler S-2021.06-SP4 performed synthesis, retiming, DesignWare architecture selection, incremental mapping, and report generation.

Tool requirements

RTL simulation requires Questa/ModelSim. Synthesis requires Synopsys Design Compiler, DesignWare, and the Nangate Open Cell Library.

Source provenance

The FPU starting design was supplied as a course subset derived from OpenHW Group CVFPU. Original file-level copyright, author, SPDX, and Solderpad Hardware License notices are preserved. The custom multiplier, FMA integration changes, directed-test modifications, synthesis study, and analysis form the project-specific work.